

✓ Trace Google Gemini Models in Langfuse

This notebook shows how to trace and observe Google Gemini models with Langfuse and the Google GenAI SDK.

What is Google Gemini? [Google Gemini](#) is Google's family of multimodal generative models (text, images, audio, video, code) available through the Gemini API and Vertex AI, with tiers like Flash and Pro for different speed/quality needs.

What is the Google GenAI SDK? The [Google GenAI SDK](#) is a unified client library (Python/JavaScript) that simplifies calling Gemini—handling auth (API key or ADC), streaming, tool/function calling, and safety—so you can integrate models in a few lines.

What is Langfuse? [Langfuse](#) is an open source platform for LLM observability and monitoring. It helps you trace and monitor your AI applications by capturing metadata, prompt details, token usage, latency, and more.

✓ Step 1: Install Dependencies

Before you begin, install the necessary packages in your Python environment:

```
%pip install google-genai openai langfuse openinference-instrumentation-google-genai
15.1.0, >=13.0.0 in /usr/local/lib/python3.12/dist-packages (from google-genai)
nsions<5.0.0, >=4.11.0 in /usr/local/lib/python3.12/dist-packages (from google-genai)
1.7.0 in /usr/local/lib/python3.12/dist-packages (from openai) (1.9.0)
.4.0 in /usr/local/lib/python3.12/dist-packages (from openai) (0.11.1)
/usr/local/lib/python3.12/dist-packages (from openai) (1.3.1)
usr/local/lib/python3.12/dist-packages (from openai) (4.67.1)

hl.metadata (14 kB)
```

```

inference-semantic-conventions in /usr/local/lib/python3.12/dist-packages (from open
n /usr/local/lib/python3.12/dist-packages (from anyio<5.0.0,>=4.8.0->google-
6.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.
les>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.
.4 in /usr/local/lib/python3.12/dist-packages (from google-auth<3.0.0,>=2.14
/usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->google-g
.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0.0,>=0.28.1->go
n /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0.0,>
etadata<8.8.0,>=6.0 in /usr/local/lib/python3.12/dist-packages (from opentele
common-protos~=1.52 in /usr/local/lib/python3.12/dist-packages (from opentele
ry-exporter-otlp-proto-common==1.37.0 in /usr/local/lib/python3.12/dist-pack
ry-proto==1.37.0 in /usr/local/lib/python3.12/dist-packages (from openteleme
0,>=5.0 in /usr/local/lib/python3.12/dist-packages (from opentelemetry-proto
ypes>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,
re==2.33.2 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.0,>
ection>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3.0.
malizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.
=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.
of opentelemetry-instrumentation to determine which version is compatible wi
n-0.58b0-py3-none-any.whl.metadata (7.1 kB)
in /usr/local/lib/python3.12/dist-packages (from importlib-metadata<8.8.0,>=
0,>=0.6.1 in /usr/local/lib/python3.12/dist-packages (from pyasn1-modules>=0
1 (364 kB)
- 364.6/364.6 kB 5.7 MB/s eta 0:00:00
google_genai-0.1.8-py3-none-any.whl (23 kB)
(15 kB)
0.1.41-py3-none-any.whl (29 kB)
ions-0.1.24-py3-none-any.whl (10 kB)
linux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (88 kB)
- 88.0/88.0 kB 4.0 MB/s eta 0:00:00
0.58b0-py3-none-any.whl (33 kB)
inference-semantic-conventions, backoff, opentelemetry-instrumentation, open
.0

use-3.8.1 openinference-instrumentation-0.1.41 openinference-instrumentation

```

✓ Step 2: Configure Langfuse SDK

Next, set up your Langfuse API keys. You can get these keys by signing up for a free [Langfuse Cloud](#) account or by [self-hosting Langfuse](#). These environment variables are essential for the Langfuse client to authenticate and send data to your Langfuse project.

Also set your Google Vertex API credentials which uses Application Default Credentials (ADC) from a service account key file.

```

import os

# Get keys for your project from the project settings page
# https://cloud.langfuse.com
LANGFUSE_SECRET_KEY = "sk-1f-afada7dc-2d00-4be8-b7d5-8759a31db192"
LANGFUSE_PUBLIC_KEY = "pk-1f-c1267e7e-1106-4389-ab27-da266102c1fa"

```

```

LANGFUSE_BASE_URL = "https://cloud.langfuse.com" # EU EU region
# LANGFUSE_BASE_URL = "https://us.cloud.langfuse.com" # us US region

# Your openai key
# We use OpenAI for this demo, could easily change to other models
os.environ["GEMINI_API_KEY"] = "AIzaSyDw18J5Ze-PxABaHV_ljdf18Da8vBgr018"

```

With the environment variables set, we can now initialize the Langfuse client.

`get_client()` initializes the Langfuse client using the credentials provided in the environment variables.

```

from langfuse import get_client

# Initialise Langfuse client and verify connectivity
langfuse = get_client()

```

WARNING:langfuse:Authentication error: Langfuse client initialized without pu

✓ Step 3: OpenTelemetry Instrumentation

Use the [GoogleGenAIInstrumentor](#) library to wrap [Google GenAI SDK](#) calls and send OpenTelemetry spans to Langfuse.

```

from openinference.instrumentation.google_genai import GoogleGenAIInstrument
GoogleGenAIInstrumentor().instrument()

```

✓ Step 4: Run an Example

```

from google import genai

client = genai.Client()

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="What is Langfuse?",
)
print(response.text)

```

****Langfuse is an open-source LLM observability and evaluation platform.****

In simpler terms, it's a tool that helps developers monitor, debug, and improve their LLM applications. Think of it as the "black box recorder" or "Google Analytics" specifically designed for LLMs. Here's a breakdown of what Langfuse does and why it's important:

1. **LLM Observability & Monitoring:**
 - * **Traces:** Langfuse captures detailed "traces" of every interaction
 - * **Visual Debugging:** It provides a user interface to visualize these
 - * **Metrics & Analytics:** It aggregates data on usage, costs, latency,
2. **Debugging & Troubleshooting:**
 - * When an LLM application behaves unexpectedly (e.g., gives a nonsensical answer)
 - * It helps pinpoint issues in prompts, RAG logic, tool definitions, or
3. **Evaluation & Improvement:**
 - * **Human Feedback:** You can collect human feedback (e.g., "good answer")
 - * **Automated Evaluation:** It supports running automated evaluations (e.g., using LLM-as-judge)
 - * **Experimentation:** Langfuse facilitates A/B testing and comparing different prompts or models

Key Features & Components:

- * **SDKs:** Provides client libraries (Python, JS/TS) to easily integrate with your LLM applications
- * **Dashboard & UI:** A web interface to view traces, manage projects, analyze usage, and receive feedback
- * **Prompt Management:** Helps version and manage your prompts
- * **Open-Source:** The core platform is open-source, allowing for self-hosting

Why is it important?

Developing robust LLM applications is challenging due to the non-deterministic nature of LLM outputs.

- * Understand complex LLM behaviors.
- * Debug issues quickly and efficiently.
- * Iterate on prompts and application logic with confidence.
- * Ensure the quality and reliability of LLM applications in production.
- * Manage costs and performance.

In essence, Langfuse bridges the gap between developing LLM-powered features

```
# Streaming Example
for chunk in client.models.generate_content_stream(
    model="gemini-2.5-flash",
    contents="What is Langfuse?",
):
    print(chunk.text, end="", flush=True)
print() # newline after streaming
```

Langfuse is an **open-source** observability and evaluation platform for LLM applications.

In simpler terms, it's a tool that helps developers building applications powered by LLMs.

1. **Understand what's happening under the hood (Observability):**
 - * **Trace and Log Everything:** It records every step of an LLM call or tool use
 - * **Visualize Flows:** It presents these traces in an intuitive waterfall view
 - * **Debug Issues:** When an LLM application doesn't behave as expected, you can see exactly what happened
 - * **Monitor Performance:** Track metrics like latency, token usage, and costs
2. **Improve the quality and reliability of their applications (Evaluation):**
 - * **Human Annotation:** Allows developers or human testers to provide feedback on LLM outputs
 - * **Automated Evaluation:** Enables the use of LLMs themselves (LLM-as-judge) to evaluate outputs
 - * **Experimentation:** Facilitates A/B testing different prompts, models, or system instructions
 - * **Dataset Management:** Helps in building and managing datasets for training or evaluation

Why is Langfuse needed?

LLM applications are inherently non-deterministic and can be complex. Traditi

- * Make the black box of LLM interactions more transparent.
- * Systematically test and iterate on prompt engineering and application log
- * Ensure the application performs reliably and cost-effectively in producti

Key Features/Benefits at a Glance:

- * ****End-to-end Tracing:**** From user input to final output, including all in
- * ****Detailed Logging:**** Captures prompts, responses, metadata, and more.
- * ****Cost & Latency Monitoring:**** Keep an eye on operational expenses and pe
- * ****Human & Automated Evaluation:**** Tools to measure and improve output qua
- * ****Experimentation & A/B Testing:**** Compare different configurations.
- * ****Open Source:**** Provides flexibility for self-hosting and community cont
- * ****SDKs & Integrations:**** Works with popular frameworks like LangChain and

In essence, Langfuse acts like a "flight recorder" and "quality assurance lab

✓ View Traces in Langfuse

After executing the application, navigate to your Langfuse Trace Table. You will find detailed traces of the application's execution, providing insights into the agent conversations, LLM calls, inputs, outputs, and performance metrics.

The screenshot displays the Langfuse web interface. The top navigation bar shows 'PROD-EU', 'langfuse-dev', 'Team', 'docs-examples', and 'Traces'. The main header area includes a search bar, a 'Trace' button, and a breadcrumb trail: 'GenerateContent: 9f2f0fe0228fd81a9fe75882934b384a'. Below this, a table lists traces with columns for ID, Latency, Env, Cost, and Prompt. The selected trace is 'GenerateContent' with ID '9f2f0fe0228fd81a9fe75882934b384a', Latency '12.03s', Env 'default', Cost '\$0.005139', and Prompt '6 prompt -> 2055 completion (Σ 2061)'. The trace details panel on the right shows the input and output. The input is a JSON object with 'model' set to 'gemini-2.5-flash' and 'contents' set to 'What is Langfuse?'. The output is a JSON object with 'sdk_http_response' set to '1 items' and 'candidates' set to '1 items'. The 'candidates' array contains a single object with 'content' set to '2 items' and 'parts' set to '1 items'. The 'parts' array contains a single object with 'text' set to 'Langfuse is an open-source observability and evaluation platform specifically designed for Large Language Model (LLM) applications. In essence, it acts as a "flight recorder" or "black box" for your AI applications, giving you deep insights into how they are performing in real-time and helping you iterate and improve them. Here's a breakdown of what Langfuse is and why it's important:'. The output is formatted as JSON.

[View trace in Langfuse](#)

<https://cloud.langfuse.com/project/cloramnkj0002jz088vzn1ja4/traces/9f2f0fe0228fd81a9fe75882934b384a?timestamp=2025-08-01T13%3A22%3A00.147Z&display=details&observation=b7a63ca7e1d083bc>

